

Automated CI/CD Pipeline with Docker and Kubernetes

End-to-End DevOps Automation using GitHub, Jenkins, Maven, Ansible, Docker, DockerHub and Kubernetes

1. Project Introduction

This project demonstrates an end-to-end DevOps automation workflow for building, containerizing and deploying a web application using GitHub, Jenkins, Maven, Ansible, Docker, DockerHub and Kubernetes.

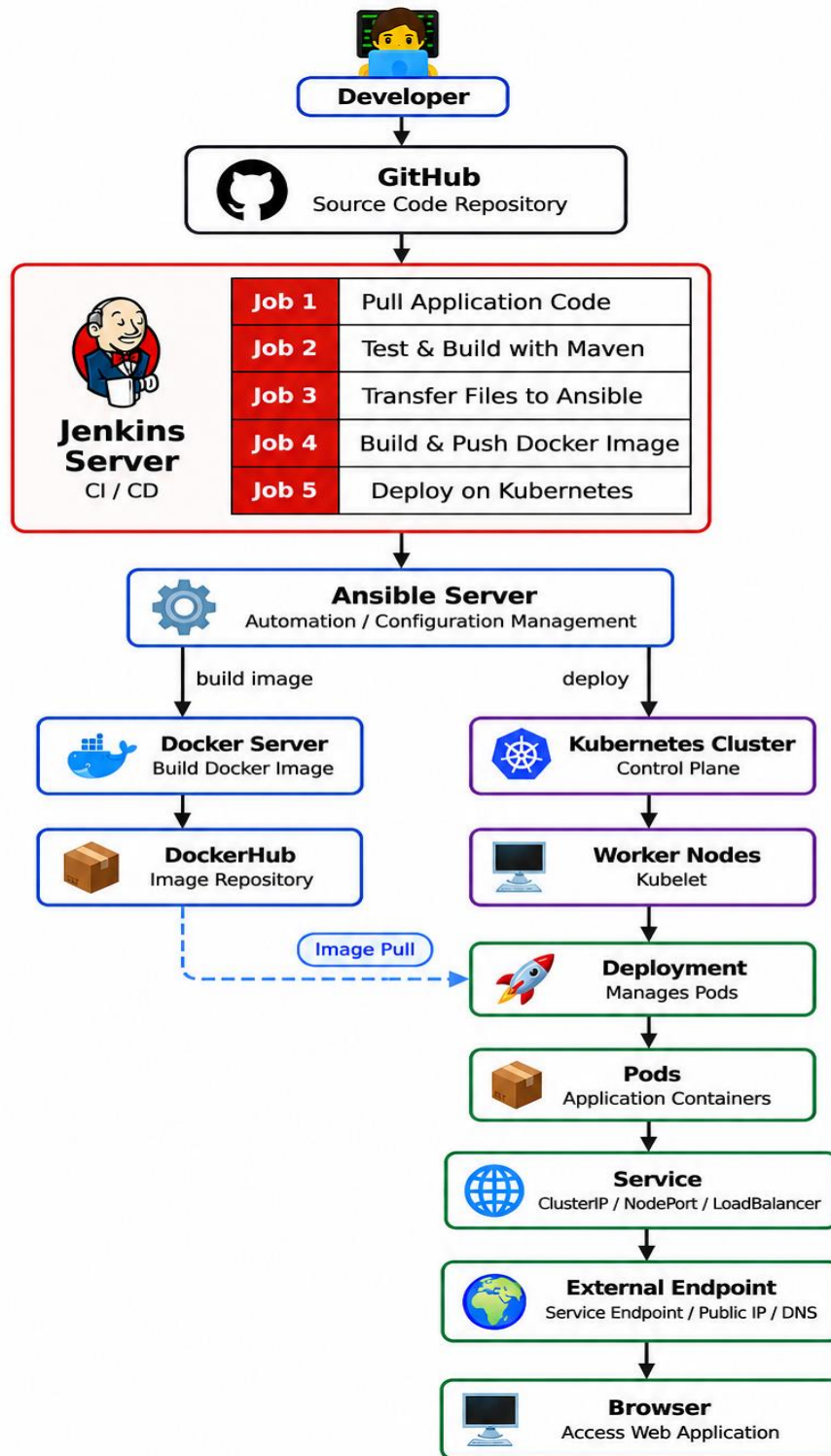
The application code is pushed to GitHub, after which Jenkins automates the CI/CD process. Jenkins pulls and builds the application using Maven, while Ansible automates tasks between the Docker and Kubernetes environments.

Docker creates the application image and pushes it to DockerHub. Finally, Kubernetes pulls the image from DockerHub, deploys the application using Pods and a Service, and makes it accessible through the worker node's public IP address.

2. Technologies Used

Technology	Purpose
Git	Version control
GitHub	Source-code repository
Jenkins	CI/CD automation
Maven	Application build and testing
Ansible	Server automation and configuration
Docker	Application containerization
DockerHub	Container image registry
Kubernetes	Container orchestration
Linux	Server operating system
Bash/Shell	Automation and server commands
Dockerfile	Docker image creation instructions
Kubernetes YAML	Application deployment configuration
Nginx	Web server/container base image
HTML	Sample web application

3. Project Architecture



4. Project Objective

The main objectives of this project are:

1. Store application source code in GitHub.
 2. Automate application deployment using Jenkins.
 3. Pull application source code automatically from GitHub.
 4. Test and build the application using Maven.
 5. Transfer application and deployment files using Jenkins.
 6. Use Ansible for automation between servers.
 7. Build a Docker image from the application.
 8. Push the Docker image to DockerHub.
 9. Create and configure a Kubernetes cluster.
 10. Deploy the application using Kubernetes Deployment.
 11. Expose the application using a Kubernetes Service.
 12. Make the application accessible through a public IP address.
 13. Demonstrate an end-to-end CI/CD and container orchestration workflow.
-

5. Step-by-Step Project Implementation

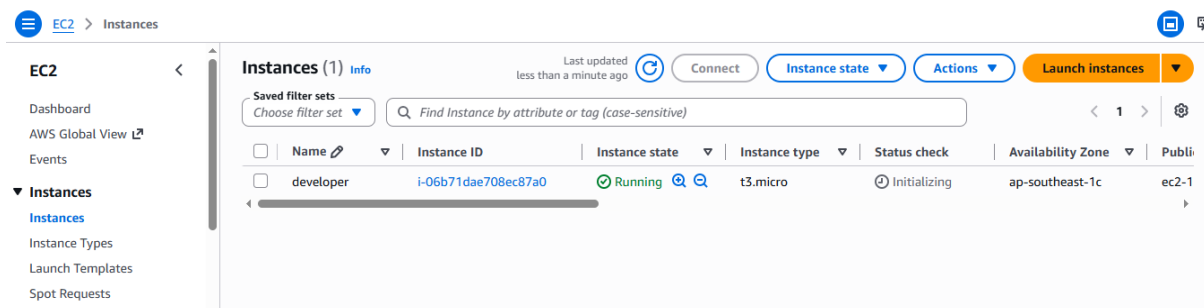
Step 1: Launch Developer Server

The first step of the project is to launch the developer server.

The developer server is used to create and maintain the application source code and project-related files.

The required application files are created on this server.

The developer server acts as the starting point of the complete DevOps workflow.



Step 2: Developer Creates Application Files and Pushes Them into Git

After launching the developer server, the developer creates the application source code and required project files.

The application source code contains the website or application files. The additional project files are used during the build, containerization and deployment process.

The files are added to Git using:

`git add .`

A commit is then created:

`git commit -m "Add application files"`

The repository is then connected to GitHub.

The basic Git workflow is:

Create Files



`git add`



`git commit`



`git push`

```
create mode 100644 webapp/src/main/webapp/assets/js/lightbox.js
create mode 100644 webapp/src/main/webapp/assets/js/owl-carousel.js
create mode 100644 webapp/src/main/webapp/assets/js/slick-slider.js
create mode 100644 webapp/src/main/webapp/assets/js/tabs.js
create mode 100644 webapp/src/main/webapp/assets/js/video.js
create mode 100644 webapp/src/main/webapp/index.html
create mode 100644 webapp/src/main/webapp/index.jsp
create mode 100644 webapp/src/main/webapp/meeting-details.html
create mode 100644 webapp/src/main/webapp/meetings.html
create mode 100644 webapp/src/main/webapp/vendor/bootstrap/css/bootstrap.min.css
s
create mode 100644 webapp/src/main/webapp/vendor/bootstrap/js/bootstrap.min.js
create mode 100644 webapp/src/main/webapp/vendor/jquery/jquery.js
create mode 100644 webapp/src/main/webapp/vendor/jquery/jquery.min.js
create mode 100644 webapp/src/main/webapp/vendor/jquery/jquery.min.map
create mode 100644 webapp/src/main/webapp/vendor/jquery/jquery.slim.js
create mode 100644 webapp/src/main/webapp/vendor/jquery/jquery.slim.min.js
create mode 100644 webapp/src/main/webapp/vendor/jquery/jquery.slim.min.map
[root@dev1 punegit]# git remote add origin https://github.com/diptimokate/devops
-project-application.git
[root@dev1 punegit]# git push https://ghp_hzavSsjxIPxUD6HgWHkoOgOtScVl7Rlk7Ry8@g
ithub.com:diptimokate/devops-project-application.git
Enumerating objects: 112, done.
Counting objects: 100% (112/112), done.
Delta compression using up to 2 threads
Compressing objects: 100% (96/96), done.
Writing objects: 100% (112/112), 7.51 MiB | 4.55 MiB/s, done.
Total 112 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/diptimokate/devops-project-application.git
 * [new branch]      master -> master
[root@dev1 punegit]#
```

Step 3: Files Are Successfully Pushed to GitHub

After committing the files, they are pushed to the GitHub repository.

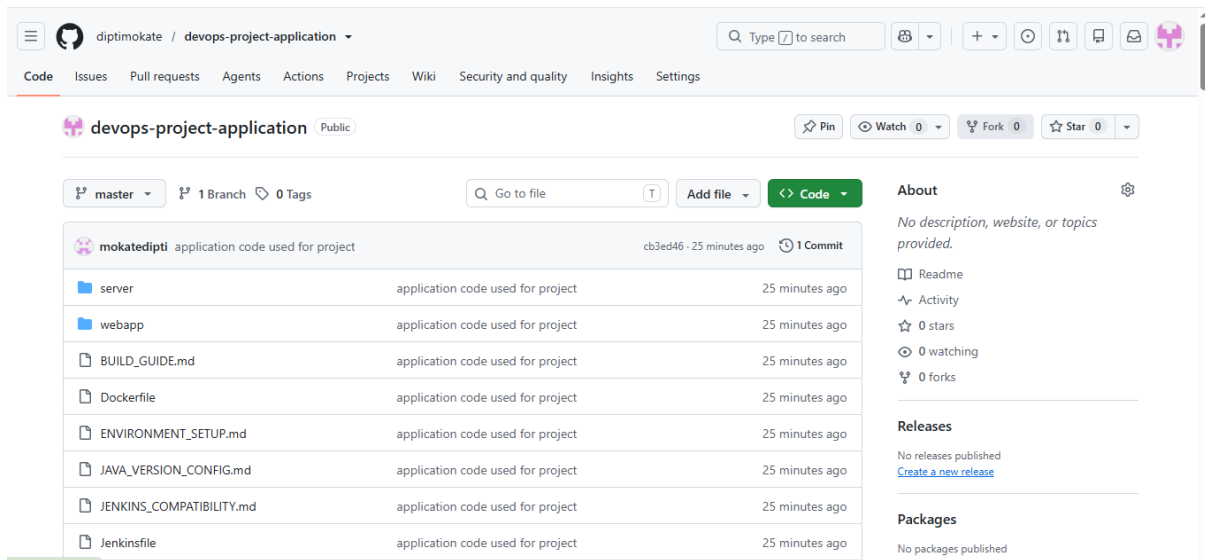
Example:

```
git push -u origin main
```

The GitHub repository now contains the application source code and required project files.

The repository acts as the centralized source-code repository for the complete project.

Jenkins uses this GitHub repository to pull the latest application code.



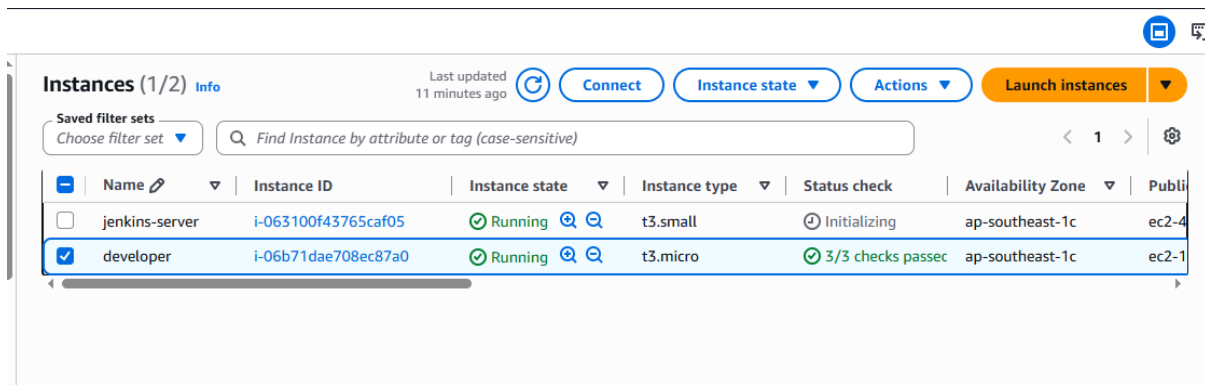
Step 4: Launch Jenkins Server

The next step is to launch the Jenkins server.

Jenkins is used as the automation and CI/CD server for the project.

Jenkins performs different tasks throughout the project, including:

- Pulling application code from GitHub.
- Testing and building the application.
- Transferring required files to Ansible.
- Creating the Docker image.
- Pushing the Docker image to DockerHub.
- Deploying the application on Kubernetes.



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IP
jenkins-server	i-063100f43765caf05	Running	t3.small	Initializing	ap-southeast-1c	ec2-4...
developer	i-06b71dae708ec87a0	Running	t3.micro	3/3 checks passed	ap-southeast-1c	ec2-1...

Step 5: Creating the First Jenkins Job – Pull Application Code from GitHub

The first Jenkins project is created to pull the application source code from GitHub.

The basic workflow is:

GitHub

↓

Jenkins

↓

Application Source Code

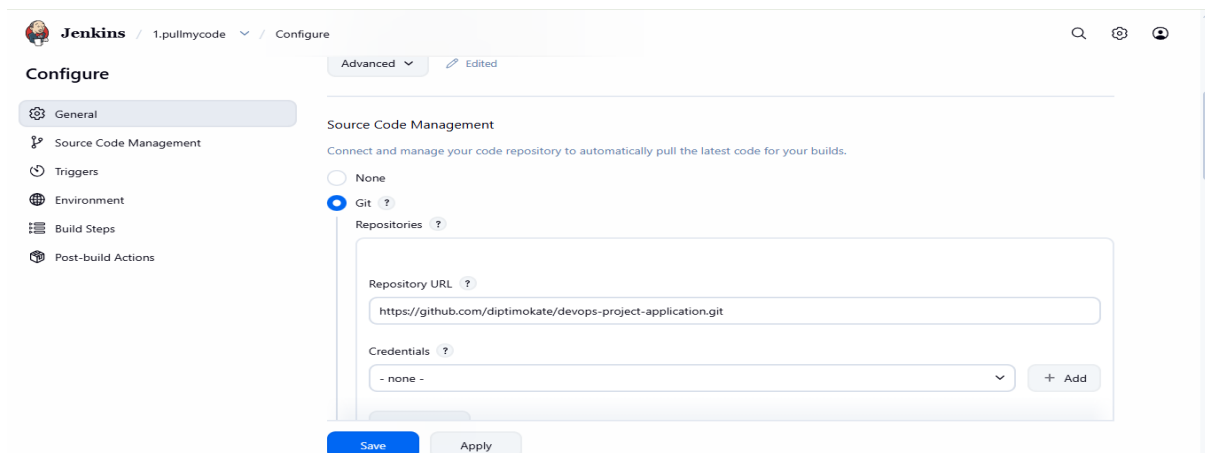
↓

Jenkins Workspace

Jenkins connects to the GitHub repository and pulls the latest version of the application source code.

The application files are stored in the Jenkins workspace after the successful build.

The first Jenkins job ensures that the latest source code is available for the next stages of the CI/CD pipeline.



Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git

Repositories

Repository URL

https://github.com/diptimokate/devops-project-application.git

Credentials

- none -

Save Apply

The screenshot shows the Jenkins dashboard. On the left, there are links for 'New Item', 'Build History', 'Build Queue' (showing 'No builds in the queue.'), and 'Build Executor Status' (showing '0/2'). The main area displays a table of build jobs. The first job, '1.pullmycode', is in a successful state, indicated by a green checkmark icon. The table headers are: S, W, Name, Last Success, Last Failure, and Last Duration. The job '1.pullmycode' has a last success time of '1 min 43 sec' and a last duration of '5.5 sec'. Below the table, there are icons for 'Icon', 'S', 'M', and 'L'.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	1.pullmycode	1 min 43 sec	N/A	5.5 sec

Step 6: Jenkins Successfully Pulls the Application Source Code

After the first Jenkins job is executed successfully, the application source code is pulled from the GitHub repository.

The successful build confirms that Jenkins can successfully connect to GitHub and access the repository.

The application code is now available inside the Jenkins workspace.

The source code can be used for the testing and build stage.

The screenshot shows a terminal window with the following output:

```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
Last login: Tue Aug 18 16:05:05 2026 from 103.174.141.244
[ec2-user@jenkins ~]$ sudo su
[root@jenkins ec2-user]# cd
[root@jenkins ~]# ls /
bin  dev  home  lib64  media  opt  registration  run  srv  tmp  var
boot  etc  lib  local  mnt  proc  root  sbin  sys  usr
[root@jenkins ~]# cd /registration/
[root@jenkins registration]# ls
BUILD_GUIDE.md          Jenkinsfile              regapp-service.yml
Dockerfile              README.md                server
ENVIRONMENT_SETUP.md    RUNTIME_VERIFICATION_REPORT.md  validate-project.sh
JAVA_VERSION_CONFIG.md  pom.xml                  webapp
JENKINS_COMPATIBILITY.md  regapp-deploy.yml
[root@jenkins registration]#
```

Step 7: Install Git and Maven for Application Build

Git and Maven are required for the application build process.

Git is used to manage and pull the application source code.

Maven is used for:

- Managing application dependencies.
- Testing the application.
- Building the application.
- Creating the application artifact.

The Jenkins server is configured with the required Git and Maven environment.

Example Maven commands:

```
mvn clean test
```

For creating the application build:

```
mvn clean package
```

The exact build command depends on the application.

```
apache-maven-3.9.16/lib/maven-repository-metadata-3.9.16.jar
apache-maven-3.9.16/lib/maven-core-3.9.16.jar
apache-maven-3.9.16/lib/plexus-cipher-2.0.jar
apache-maven-3.9.16/lib/maven-resolver-api-1.9.27.jar
apache-maven-3.9.16/lib/plexus-sec-dispatcher-2.0.jar
apache-maven-3.9.16/lib/maven-shared-utils-3.4.2.jar
apache-maven-3.9.16/lib/org.eclipse.sisu.plexus-1.0.0.jar
apache-maven-3.9.16/lib/wagon-file-3.5.3.jar
apache-maven-3.9.16/lib/maven-resolver-transport-file-1.9.27.jar
apache-maven-3.9.16/lib/commons-codec-1.21.0.jar
apache-maven-3.9.16/lib/maven-compat-3.9.16.jar
apache-maven-3.9.16/lib/plexus-component-annotations-2.2.0.jar
apache-maven-3.9.16/lib/javax.inject-1.jar
apache-maven-3.9.16/lib/maven-resolver-util-1.9.27.jar
apache-maven-3.9.16/lib/asm-9.9.1.jar
apache-maven-3.9.16/lib/failureaccess-1.0.3.jar
apache-maven-3.9.16/lib/wagon-http-shared-3.5.3.jar
apache-maven-3.9.16/lib/httpclient-4.5.14.jar
apache-maven-3.9.16/lib/gson-2.13.2.jar
apache-maven-3.9.16/lib/maven-plugin-api-3.9.16.jar
apache-maven-3.9.16/lib/commons-cli-1.11.0.jar
[root@jenkins opt]# ls
apache-maven-3.9.16  apache-maven-3.9.16-bin.tar.gz  aws
[root@jenkins opt]#
```

Step 8: Creating the Second Jenkins Job – Test and Build the Application

The second Jenkins job is created to perform the Continuous Integration process.

The application code pulled from GitHub is tested and built using Maven.

The basic workflow is:

Application Source Code



Maven Test



Maven Build



Application Artifact

The Jenkins job automatically performs the required build operations.

If the build is successful, the application is ready for the next deployment stages.

The image shows two screenshots of the Jenkins web interface. The top screenshot is the 'Configure' page for a job named '2.test and buildmycode'. The 'Build Steps' tab is selected in the left sidebar. The configuration shows a build step named 'Invoke top-level Maven targets' with 'Maven Version' set to 'mvn' and 'Goals' set to 'clean install package'. The bottom screenshot is the 'Build History' page for the same job. It shows a table of build records with columns for status, icon, name, last success, last failure, and last duration. Two builds are listed: '1.pullmycode' and '2.test and buildmycode', both of which are successful.

Jenkins Configuration Screenshot:

Configure 2.test and buildmycode / Configure

Build Steps

Invoke top-level Maven targets

Maven Version: mvn

Goals: clean install package

Jenkins Build History Screenshot:

Build Queue: No builds in the queue.

Build Executor Status: 0/2

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	1.pullmycode	32 sec #2	N/A	0.73 sec
✓	☀	2.test and buildmycode	23 sec #1	N/A	19 sec

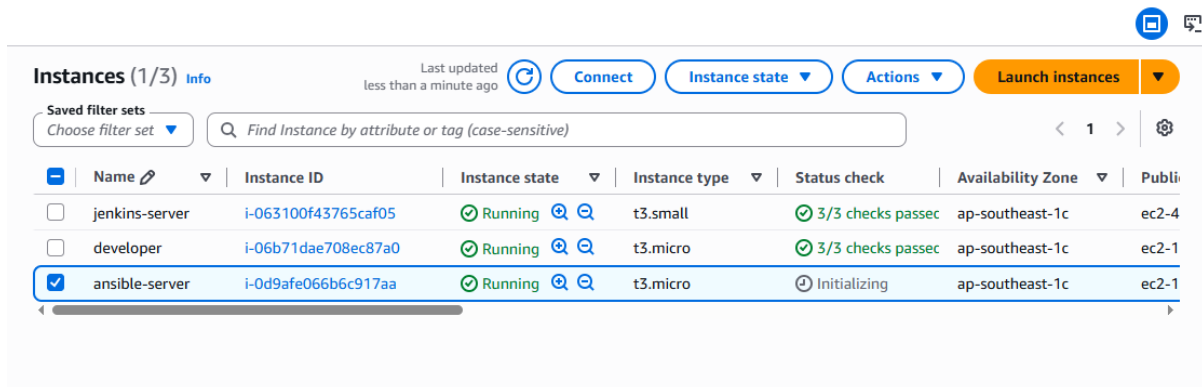
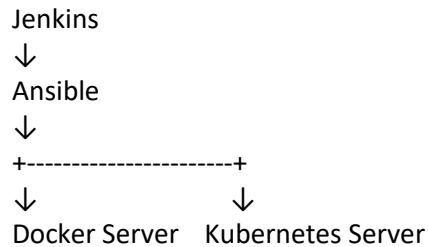
Step 9: Launch Ansible Server

The next step is to launch the Ansible server.

The Ansible server is used for server automation and configuration management.

The Ansible server is responsible for automating tasks on the Docker server and Kubernetes environment.

The general workflow is:



The screenshot shows the AWS Management Console 'Instances' page. It displays a table of EC2 instances. The 'ansible-server' instance is selected and highlighted in blue. The table columns include Name, Instance ID, Instance state, Instance type, Status check, Availability Zone, and Public IP. The 'ansible-server' instance is in the 'Initializing' state, while the others are 'Running'.

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IP
☐	jenkins-server	i-063100f43765caf05	Running	t3.small	3/3 checks passed	ap-southeast-1c	ec2-4
☐	developer	i-06b71dae708ec87a0	Running	t3.micro	3/3 checks passed	ap-southeast-1c	ec2-1
☒	ansible-server	i-0d9afe066b6c917aa	Initializing	t3.micro	Initializing	ap-southeast-1c	ec2-1

Step 10: Creating the Third Jenkins Job – Transfer Files to Ansible Server

The third Jenkins job is created to transfer the required application and deployment files to the Ansible server.

The Jenkins job sends the required files to the Ansible server.

The transferred files may include:

- Application files.
- Dockerfile.
- Ansible playbooks.
- Kubernetes Deployment YAML file.
- Kubernetes Service YAML file.

The workflow is:

Jenkins

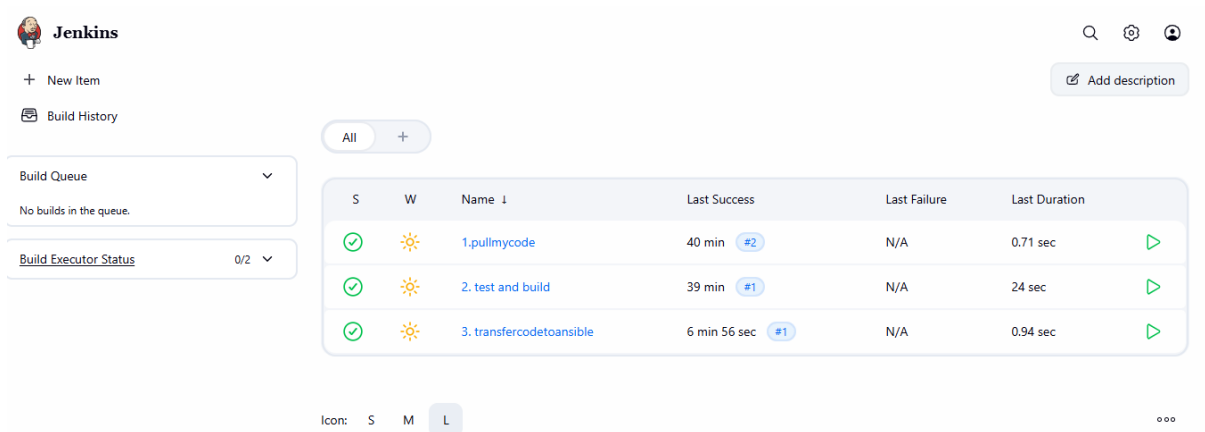
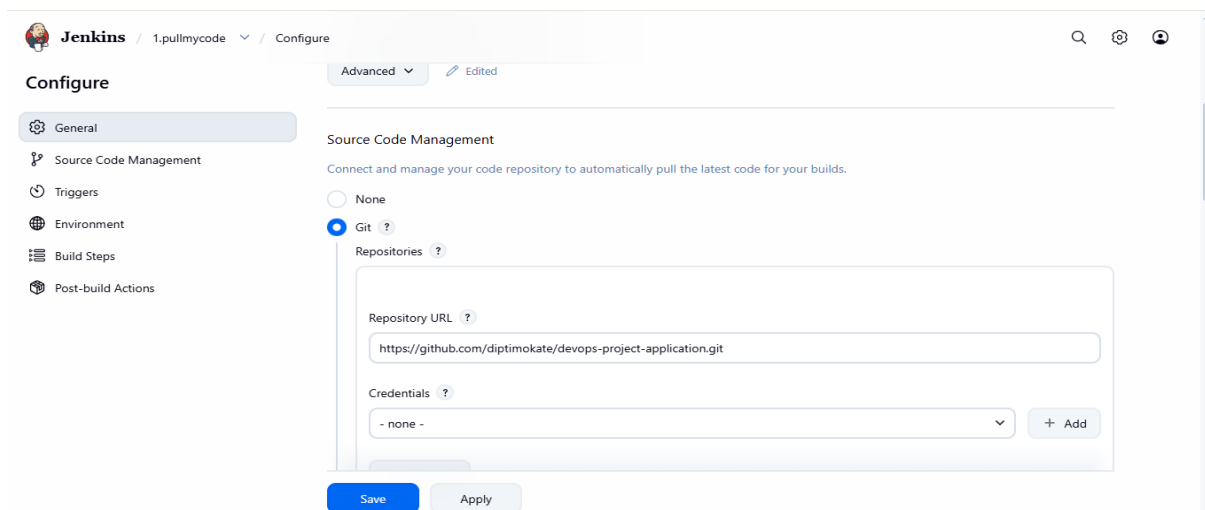


Application / Deployment Files



Ansible Server

The Ansible server now contains the required files for automation.



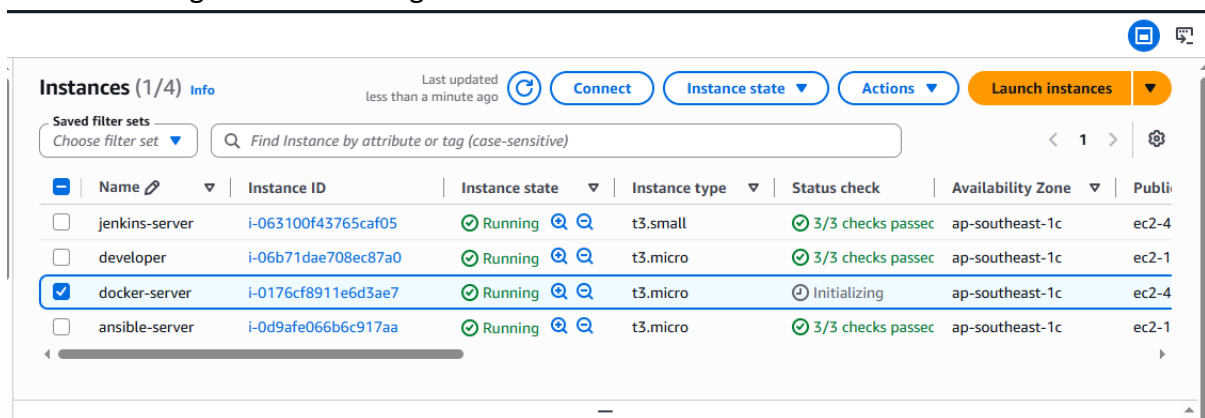
Step 11: Launch Docker Server

The next step is to launch the Docker server.

The Docker server is used to containerize the application.

Docker is responsible for:

- Receiving the application files.
- Reading the Dockerfile.
- Creating the Docker image.
- Tagging the Docker image.
- Pushing the Docker image to DockerHub.



Step 12: Create DockerHub Repository

A DockerHub repository is created to store the application container image.

DockerHub acts as a centralized container image registry.

The image created by Docker is uploaded to DockerHub.

The general workflow is:

Docker Server

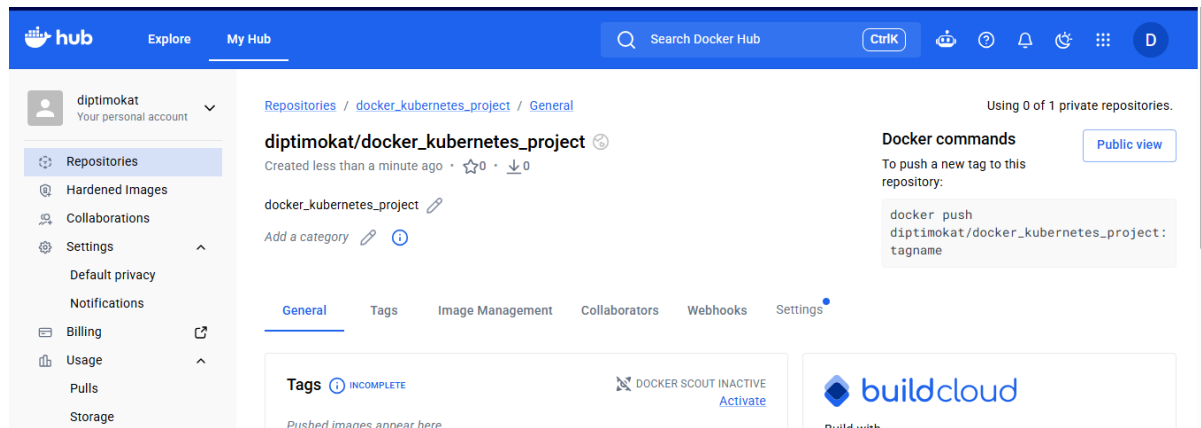
↓

Docker Image

↓

DockerHub Repository

The Kubernetes cluster later pulls the application image from DockerHub.



Step 13: Creating the Fourth Jenkins Job – Build and Push Image to DockerHub

The fourth Jenkins job is created to automate Docker image creation and upload.

The Jenkins job performs the following operations:

1. Pulls or receives the application files.
2. Uses the Dockerfile.
3. Builds the Docker image.
4. Tags the image with the DockerHub repository name.
5. Pushes the image to DockerHub.

The basic workflow is:

Application Source Code



Dockerfile



Docker Build



Docker Image

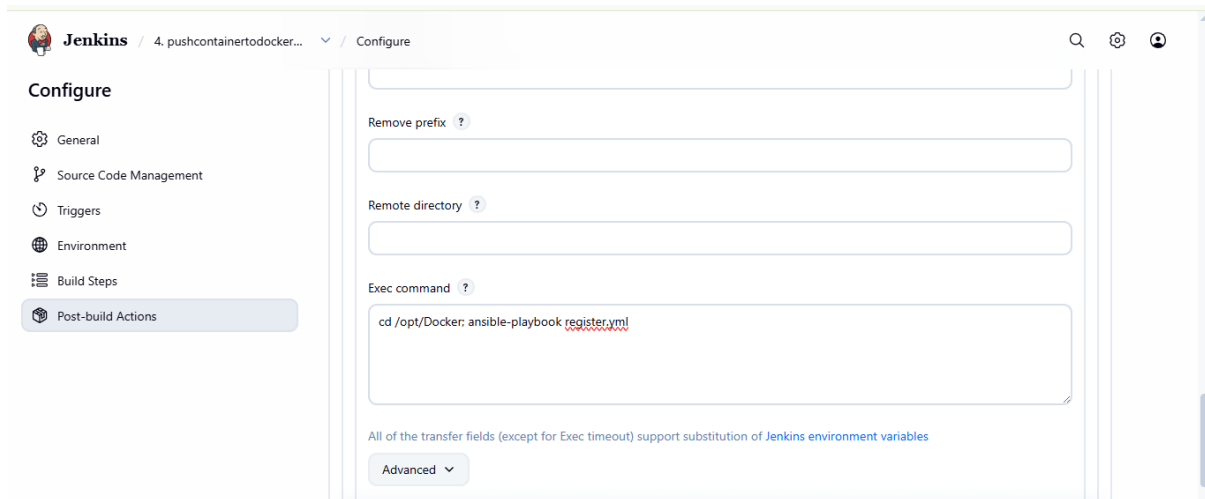


DockerHub

Example Docker build command:

```
docker build -t <dockerhub-username>/<repository-name>:latest .
```

The Docker image is then pushed to DockerHub.



Step 14: Application Docker Image Is Successfully Pushed to DockerHub

After the Docker image is successfully created, it is pushed to DockerHub.

Example login command:

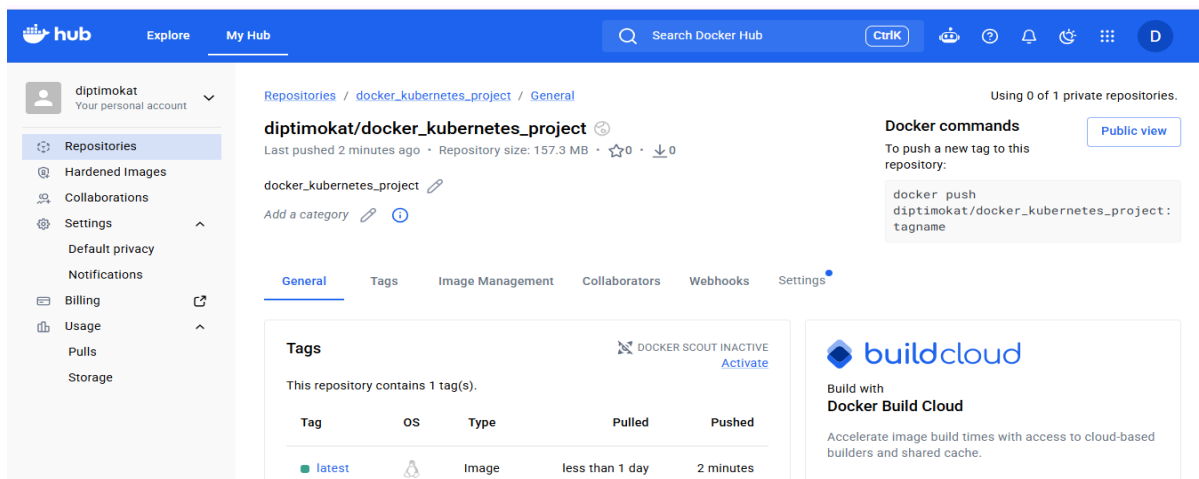
```
docker login
```

Then the image is pushed using:

```
docker push <dockerhub-username>/<repository-name>:latest
```

DockerHub now contains the application image.

The DockerHub repository acts as the container image registry from which the Kubernetes cluster can pull the application image during deployment.



Step 15: Launch Kubernetes Server

The next step is to create the Kubernetes environment.

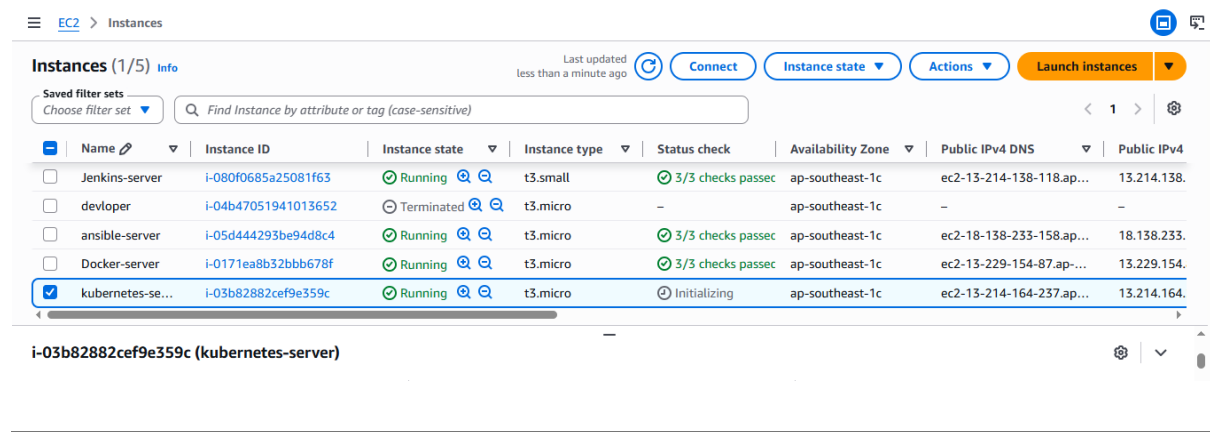
The Kubernetes cluster contains:

- Control Plane / Master Node.
- Worker Node or Nodes.

The control plane manages the Kubernetes cluster.

The worker nodes run the application Pods.

The Kubernetes environment is used to deploy and manage the containerized application.



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4
Jenkins-server	i-080f0685a25081f63	Running	t3.small	3/3 checks passed	ap-southeast-1c	ec2-13-214-138-118.ap...	13.214.138.
developer	i-04b47051941013652	Terminated	t3.micro	-	ap-southeast-1c	-	-
ansible-server	i-05d444293be94d8c4	Running	t3.micro	3/3 checks passed	ap-southeast-1c	ec2-18-138-233-158.ap...	18.138.233.
Docker-server	i-0171ea8b32bb678f	Running	t3.micro	3/3 checks passed	ap-southeast-1c	ec2-13-229-154-87.ap...	13.229.154.
kubernetes-se...	i-03b82882cef9e359c	Initializing	t3.micro	Initializing	ap-southeast-1c	ec2-13-214-164-237.ap...	13.214.164.

Step 16: Ansible Connects to Docker Server and Kubernetes Environment

After configuring the Ansible server, Ansible is used to communicate with the required servers.

The Ansible server connects to:

- Docker Server.
- Kubernetes Control Plane.
- Kubernetes Worker Nodes.

The workflow is:

Ansible Server

↓

+-----+

↓

Docker Server

↓

Build Image

↓

Kubernetes Cluster

↓

Deploy Application

Ansible automates the required remote operations.

```
[itadmin@ansible Docker]$ ansible all -m ping
[WARNING]: Platform linux on host kubernetes is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.15/reference\_appendices/interpreter\_discovery.html for more information.
kubernetes | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Platform linux on host docker_server is using the discovered Python interpreter at
/usr/bin/python3.9, but future installation of another Python interpreter could change the
meaning of that path. See https://docs.ansible.com/ansible-
core/2.15/reference\_appendices/interpreter\_discovery.html for more information.
docker_server | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[itadmin@ansible Docker]$
[itadmin@ansible Docker]$
```

Step 17: Kubernetes Nodes Are Ready

After the Kubernetes cluster is configured, the nodes are verified.

The following command is used:

```
kubectl get nodes
```

The Kubernetes cluster is now ready for application deployment.

```
[root@eks ~]# kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-192-168-46-54.ap-south-1.compute.internal Ready    <none>   34m   v1.34.9-
eks-b3f9404
ip-192-168-89-225.ap-south-1.compute.internal Ready    <none>   34m   v1.34.9-
eks-b3f9404
[root@eks ~]#
```

Step 18: Create Kubernetes Deployment and Service Files

The Kubernetes Deployment YAML file is used to deploy the application.

The Deployment file defines:

- Application name.
- Number of replicas.
- Docker image.
- Container configuration.
- Application labels.

The Kubernetes Service YAML file is used to expose the application.

The Service defines:

- Service name.
- Service type.
- Application selector.
- Port configuration.
- Target port.

The general workflow is:

DockerHub Image

↓

Kubernetes Deployment

↓

Application Pods

↓

Kubernetes Service

↓

Browser Access

Step 19: Kubernetes Creates Application Pods

The Kubernetes Deployment file is applied to the cluster.

Example:

```
kubectl apply -f deployment.yml
```

The Service file is then applied:

```
kubectl apply -f service.yml
```

Kubernetes pulls the application image from DockerHub and creates the required Pods.

The Pods can be verified using:

```
kubectl get pods
```

The Service can be verified using:

```
kubectl get svc
```

Step 20: Creating the Fifth Jenkins Job – Deploy Application on Kubernetes

The fifth Jenkins job is created to automate the application deployment process.

The Jenkins job triggers the Kubernetes deployment process.

The application image is pulled from DockerHub and deployed on the Kubernetes cluster.

The workflow is:

DockerHub

↓

Application Image

↓

Jenkins Job

↓

Kubernetes

↓

Deployment

↓

Pods

↓

Service

The application is now deployed on the Kubernetes environment.

 Jenkins

5. deploymycode

Configure

Remove prefix ?

Remote directory ?

Exec command ?

cd /opt/Docker; ansible-playbook deploymycode.yml

All of the transfer fields (except for Exec timeout) support substitution of Jenkins environment variables

Advanced

Step 21: Verify Kubernetes Pods and Service

After successful deployment, the Kubernetes resources are verified.

Check the Pods:

```
kubectl get pods
```

Check the Deployment:

```
kubectl get deployment
```

Check the Service:

```
kubectl get svc
```

The Pods should be in the Running state.

The Service should expose the application correctly.

```
root@eks:~  
[root@eks ~]# kubectl get nodes  
NAME                                STATUS    ROLES    AGE    VERSION  
ip-192-168-16-26.ap-south-1.compute.internal Ready    <none>   3m40s   v1.34.9-eks-b3f9404  
ip-192-168-46-54.ap-south-1.compute.internal Ready    <none>   91m     v1.34.9-eks-b3f9404  
ip-192-168-89-225.ap-south-1.compute.internal Ready    <none>   91m     v1.34.9-eks-b3f9404  
[root@eks ~]# kubectl get pods  
NAME                                READY    STATUS    RESTARTS   AGE  
sevenmentor-regapp-76c65d855f-6kpdj 1/1      Running   0           19m  
[root@eks ~]# kubectl get svc  
NAME            TYPE          CLUSTER-IP    EXTERNAL-IP                                PORT(S)          AGE  
kubernetes      ClusterIP     10.100.0.1     <none>                                     443/TCP          97m  
sevenmentor-service LoadBalancer 10.100.17.226 aldda4c20bbfc4d4aa7e04101a54fd78-1417304279.ap-south-1.elb.amazonaws.com 8080:32312/TCP   19m  
[root@eks ~]#
```

Step 22: Application Accessible Through Browser

The final step is to access the deployed application.

Once the Kubernetes Deployment and Service are successfully created, the application becomes accessible through the Kubernetes Service.

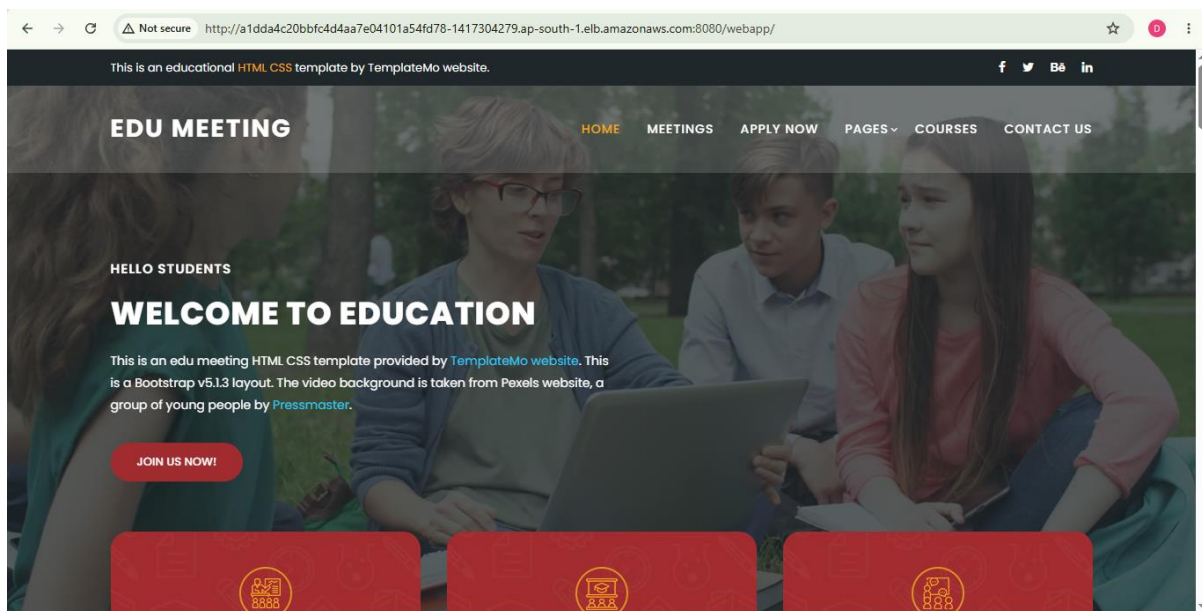
For a NodePort service, the worker node public IP and NodePort are used.

Example:

`http://<WORKER_PUBLIC_IP>:<NODEPORT>`

The exact public IP address and port depend on the Kubernetes Service configuration.

The successful browser access confirms that the complete DevOps CI/CD pipeline is working correctly.



Step 23: All jobs and complete project view

**Jenkins**

+ New Item

Build History

Build Queue

No builds in the queue.

Build Executor Status

(0 of 2 executors busy)

All

+

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	1.pullmycode	2 hr 41 min #3	N/A	0.64 sec
✓	☀	2.test and buildmycode	1 min 23 sec #3	N/A	8.2 sec
✓	☀	3. transfercodetoansible	1 min 5 sec #3	N/A	0.68 sec
✓	☀	4. pushcontainertodockerhub	55 sec #3	N/A	29 sec
✓	☀	5. deploymycode	25 min #3	N/A	8.2 sec

Icon: S M L

🔍 ⚙️ 👤

Add description

**Jenkins**

/ complete project flow

🔍 ⚙️ 👤

Build Pipeline

▶ History ⚙️ Configure + Add Step 🗑 Delete 🔧 Manage

Pipeline #4

#4 1.pullmycode

Aug 25, 2026 5:37:25 PM

0.61 sec

admin

→

#4 2.test and buildmycode

Aug 25, 2026 5:37:32 PM

7.9 sec

→

#4 3. transfercodetoansible

Aug 25, 2026 5:37:47 PM

0.63 sec

→

#4 4. pushcontainertodockerhub

Aug 25, 2026 5:37:57 PM

32 sec

→

#4 5. deploymycode

Aug 25, 2026 5:38:37 PM

8.7 sec

6. Complete Project Flow

The complete project can be summarized as:

1 : Launch Developer Server



STEP 2 : Create Application Files



STEP 3 : Push Files to GitHub



STEP 4 : Launch Jenkins Server



STEP 5 : Create First Jenkins Job to Pull Code



STEP 6 : Jenkins Successfully Pulls Code



STEP 7 : Configure Git and Maven



STEP 8 : Test and Build Application



STEP 9 : Launch Ansible Server



STEP 10 : Transfer Files to Ansible



STEP 11 : Launch Docker Server



STEP 12 : Create DockerHub Repository



STEP 13 : Build Docker Image Through Jenkins



STEP 14 : Push Docker Image to DockerHub



STEP 15 : Launch Kubernetes Cluster



STEP 16 : Configure Ansible Connectivity



STEP 17 : Verify Kubernetes Nodes



STEP 18 : Create Deployment and Service Files



STEP 19 : Create Application Pods



STEP 20 : Deploy Application Through Fifth Jenkins Job



STEP 21 : Verify Pods and Service



STEP 22 : Access Application Through Browser

7. Final Project Result

The project successfully demonstrates an end-to-end automated DevOps deployment workflow.

The application starts from the developer server and is pushed to GitHub.

Jenkins then automates the different CI/CD stages. The application source code is pulled from GitHub, tested and built using Maven, and transferred for further automation.

Docker is used to create the application container image. The image is stored in DockerHub.

Ansible is used to automate operations between the required servers and environments.

The application image stored in DockerHub is then deployed on the Kubernetes cluster.

Kubernetes creates and manages the application Pods and Service.

Finally, the deployed application becomes accessible through the Kubernetes worker node public IP address and Service port.

Therefore, the complete project demonstrates the integration of:

Git + GitHub + Jenkins + Maven + Ansible + Docker + DockerHub + Kubernetes

into one automated DevOps workflow.

8. Conclusion

This project provides practical implementation of important DevOps concepts including source-code management, Continuous Integration, CI/CD automation, application building, configuration management, containerization, container image registry management and container orchestration.

The project demonstrates how different DevOps tools can work together in one automated deployment pipeline.

The application starts from the developer and moves through GitHub, Jenkins, Maven, Ansible, Docker, DockerHub and Kubernetes.

Jenkins automates the CI/CD process, Maven performs application build operations, Ansible manages automation tasks, Docker containerizes the application, DockerHub stores the container image and Kubernetes deploys and manages the application.

The final result is a working application deployed on a Kubernetes environment and accessible through the Kubernetes Service.

This project demonstrates the integration of:

Developer



GitHub



Jenkins



Maven



Ansible



Docker



DockerHub



Kubernetes



Browser